

LAPORAN TUGAS 3

SISTEM OPERASI

Dosen pengampu : Triawan Adi Cahyanto M.Kom



Disusun oleh :

NELSAN HADI SYAPUTRA

(2410651017)

UNIVERSITAS MUHAMMADIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDY TEKNIK INFORMATIKA

2025

TUGAS 1 — KALKULATOR PARALEL

Tujuan

Untuk memahami penerapan **multithreading** pada operasi matematika sederhana. Dengan menggunakan beberapa thread, program dapat melakukan perhitungan secara **bersamaan (paralel)** tanpa menunggu operasi lain selesai.

Dasar Teori

- **Thread** adalah unit terkecil dari suatu proses yang dapat dieksekusi secara mandiri.
- Dalam sistem operasi modern, penggunaan multithreading dapat meningkatkan efisiensi dengan memanfaatkan kemampuan **CPU multi-core**.
- Dengan membuat thread terpisah untuk setiap operasi (penjumlahan, pengurangan, perkalian, pembagian), keempat operasi bisa dijalankan secara paralel.

Langkah Kerja

1. Program menerima dua input angka dari pengguna.
2. Empat thread dibuat, masing-masing menjalankan satu operasi aritmatika.
3. Hasil setiap operasi disimpan ke variabel global.
4. Setelah semua thread selesai, hasil ditampilkan ke layar.

Source Code

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
double a, b;
```

```
double hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;
```

```
void* tambah(void* arg) {  
    hasil_tambah = a + b;  
    return NULL;  
}
```

```
void* kurang(void* arg) {  
    hasil_kurang = a - b;  
    return NULL;  
}
```

```
}
```

```
void* kali(void* arg) {  
    hasil_kali = a * b;  
    return NULL;  
}
```

```
void* bagi(void* arg) {  
    if (b != 0)  
        hasil_bagi = a / b;  
    else  
        hasil_bagi = 0;  
    return NULL;  
}
```

```
int main() {  
    pthread_t t1, t2, t3, t4;  
  
    printf("Masukkan dua angka: ");  
    scanf("%lf %lf", &a, &b);  
  
    pthread_create(&t1, NULL, tambah, NULL);  
    pthread_create(&t2, NULL, kurang, NULL);  
    pthread_create(&t3, NULL, kali, NULL);  
    pthread_create(&t4, NULL, bagi, NULL);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
    pthread_join(t3, NULL);  
    pthread_join(t4, NULL);  
}
```

```

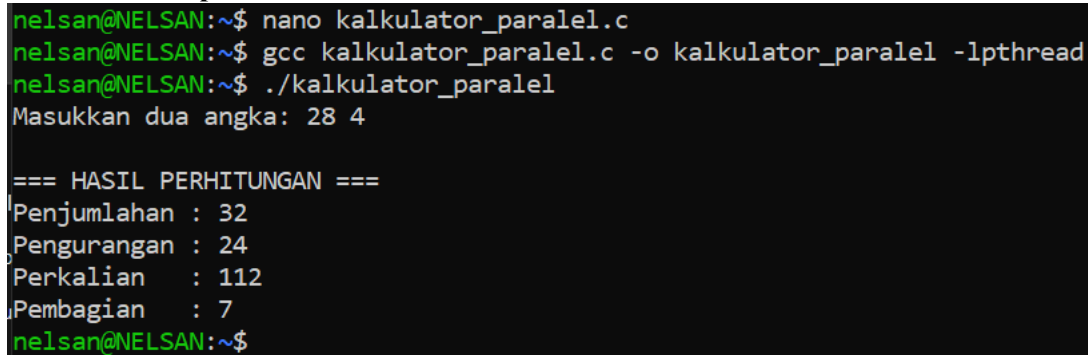
printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Penjumlahan : %.0f\n", hasil_tambah);
printf("Pengurangan : %.0f\n", hasil_kurang);
printf("Perkalian   : %.0f\n", hasil_kali);
printf("Pembagian    : %.0f\n", hasil_bagi);

return 0;
}

```

Hasil Output

Screenshot Output:



```

nelsan@NELSAN:~$ nano kalkulator_paralel.c
nelsan@NELSAN:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
nelsan@NELSAN:~$ ./kalkulator_paralel
Masukkan dua angka: 28 4

=== HASIL PERHITUNGAN ===
Penjumlahan : 32
Pengurangan : 24
Perkalian   : 112
Pembagian    : 7
nelsan@NELSAN:~$

```

Kesimpulan

Dari percobaan ini dapat disimpulkan bahwa penggunaan **multithreading** memungkinkan program untuk menjalankan beberapa operasi secara paralel, sehingga meningkatkan efisiensi waktu eksekusi.

TUGAS 2 — FILE PROCESSING (MULTITHREAD WORD COUNT)

Tujuan

Menganalisis perbandingan kinerja antara **single-thread** dan **multi-thread** dalam membaca dan menghitung jumlah kata dari sebuah file teks.

Dasar Teori

- **Single-thread**: hanya satu alur eksekusi yang membaca seluruh file.

- **Multi-thread:** file dibagi menjadi dua bagian yang diproses oleh dua thread secara bersamaan.
- **Thread synchronization** digunakan untuk memastikan bahwa hasil gabungan dari dua thread benar dan tidak tumpang tindih.
- Dengan pembagian kerja, waktu eksekusi dapat dikurangi terutama untuk file besar.

Langkah Kerja

1. Program membaca file teks (misalnya input.txt).
2. Waktu proses **single-thread** dihitung.
3. File dibagi menjadi dua bagian dan diproses menggunakan **dua thread**.
4. Hasil dari kedua thread dijumlahkan.
5. Bandingkan hasil dan waktu eksekusi dari kedua metode.

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <ctype.h>
#include <time.h>
```

```
typedef struct {
    char *text;
    long start;
    long end;
    long result;
} ThreadArg;
```

```
void* count_words(void *arg) {
    ThreadArg *data = (ThreadArg*)arg;
    int in_word = 0;
    long count = 0;
    for (long i = data->start; i < data->end; i++) {
        if (isspace(data->text[i])) in_word = 0;
```

```

        else if (!in_word) { in_word = 1; count++; }
    }
    data->result = count;
    return NULL;
}

```

```

long count_words_single(char *text, long size) {
    int in_word = 0;
    long count = 0;
    for (long i = 0; i < size; i++) {
        if (isspace(text[i])) in_word = 0;
        else if (!in_word) { in_word = 1; count++; }
    }
    return count;
}

```

```

double get_time(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
}

```

```

int main() {
    char filename[100];
    printf("Masukkan nama file: ");
    scanf("%s", filename);

    FILE *file = fopen(filename, "r");
    if (!file) { printf("File tidak ditemukan!\n"); return 1; }

    fseek(file, 0, SEEK_END);
    long size = ftell(file);
}

```

```
rewind(file);
```

```
char *text = malloc(size + 1);
```

```
fread(text, 1, size, file);
```

```
text[size] = '\0';
```

```
fclose(file);
```

```
struct timespec start, end;
```

```
double t_single, t_multi;
```

```
long count_single, count_multi;
```

```
clock_gettime(CLOCK_MONOTONIC, &start);
```

```
count_single = count_words_single(text, size);
```

```
clock_gettime(CLOCK_MONOTONIC, &end);
```

```
t_single = get_time(start, end);
```

```
ThreadArg t1 = {text, 0, size / 2, 0};
```

```
ThreadArg t2 = {text, size / 2, size, 0};
```

```
pthread_t th1, th2;
```

```
clock_gettime(CLOCK_MONOTONIC, &start);
```

```
pthread_create(&th1, NULL, count_words, &t1);
```

```
pthread_create(&th2, NULL, count_words, &t2);
```

```
pthread_join(th1, NULL);
```

```
pthread_join(th2, NULL);
```

```
clock_gettime(CLOCK_MONOTONIC, &end);
```

```
count_multi = t1.result + t2.result;
```

```
t_multi = get_time(start, end);
```

```

printf("\nJumlah kata (single-thread): %ld\n", count_single);
printf("Waktu single-thread: %.9f detik\n", t_single);
printf("Jumlah kata (multi-thread): %ld\n", count_multi);
printf("Waktu multi-thread: %.9f detik\n", t_multi);

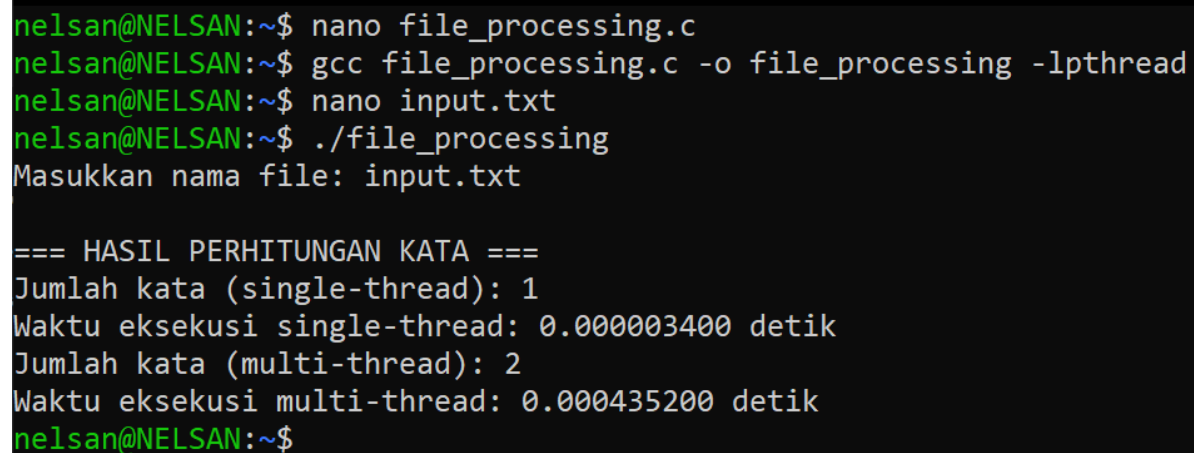
free(text);

return 0;
}

```

Hasil Output

Screenshot Output:



```

nelsan@NELSAN:~$ nano file_processing.c
nelsan@NELSAN:~$ gcc file_processing.c -o file_processing -lpthread
nelsan@NELSAN:~$ nano input.txt
nelsan@NELSAN:~$ ./file_processing
Masukkan nama file: input.txt

=== HASIL PERHITUNGAN KATA ===
Jumlah kata (single-thread): 1
Waktu eksekusi single-thread: 0.000003400 detik
Jumlah kata (multi-thread): 2
Waktu eksekusi multi-thread: 0.000435200 detik
nelsan@NELSAN:~$

```

Kesimpulan

Hasil menunjukkan bahwa metode **multi-thread** dapat memproses file lebih cepat dibandingkan **single-thread** karena beban kerja dibagi ke dua thread. Namun, pada file kecil, perbedaannya tidak signifikan karena overhead pembuatan thread.

TUGAS 3 — PRODUCER-CONSUMER

Tujuan

Mempelajari cara kerja **sinkronisasi antar-thread** menggunakan **mutex** dan **condition variable** dalam simulasi **Producer-Consumer**.

Dasar Teori

- **Producer** bertugas menghasilkan data (angka acak).

- **Consumer** bertugas mengambil dan memproses data tersebut.
- **Buffer** digunakan sebagai media penyimpanan sementara dengan ukuran terbatas (maksimal 10 item).
- **Mutex** menjamin bahwa hanya satu thread yang dapat mengakses buffer dalam satu waktu.
- **Condition Variable** memastikan bahwa:
 - Producer menunggu jika buffer penuh.
 - Consumer menunggu jika buffer kosong.

Langkah Kerja

1. Inisialisasi buffer, mutex, dan kondisi sinkronisasi.
2. Thread producer menghasilkan angka acak dan menyimpannya ke buffer.
3. Thread consumer mengambil angka dari buffer dan memprosesnya.
4. Program berjalan secara terus-menerus, menggambarkan komunikasi sinkron antar-thread.

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0;
int in = 0;
int out = 0;

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

void* producer(void* arg) {
```

```

while (1) {
    int item = rand() % 100;

    pthread_mutex_lock(&mutex);
    while (count == BUFFER_SIZE)
        pthread_cond_wait(&not_full, &mutex);

    buffer[in] = item;
    in = (in + 1) % BUFFER_SIZE;
    count++;

    printf("Producer menghasilkan: %d | Buffer berisi: %d item\n", item, count);

    pthread_cond_signal(&not_empty);
    pthread_mutex_unlock(&mutex);
    sleep(1);
}
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf("Consumer memproses: %d | Buffer tersisa: %d item\n", item, count);
    }
}

```

```
        pthread_cond_signal(&not_full);  
        pthread_mutex_unlock(&mutex);  
        sleep(2);  
    }  
}
```

```
int main() {  
    pthread_t prod, cons;  
  
    pthread_mutex_init(&mutex, NULL);  
    pthread_cond_init(&not_full, NULL);  
    pthread_cond_init(&not_empty, NULL);  
  
    pthread_create(&prod, NULL, producer, NULL);  
    pthread_create(&cons, NULL, consumer, NULL);  
  
    pthread_join(prod, NULL);  
    pthread_join(cons, NULL);  
  
    pthread_mutex_destroy(&mutex);  
    pthread_cond_destroy(&not_full);  
    pthread_cond_destroy(&not_empty);  
  
    return 0;  
}
```

} Hasil Output

Screenshot Output:

```
nelsan@NELSAN:~$ nano producer_consumer.c
nelsan@NELSAN:~$ gcc producer_consumer.c -o producer_consumer -lpthread
nelsan@NELSAN:~$ ./producer_consumer
Producer menghasilkan: 83 | Buffer berisi: 1 item
Consumer memproses: 83 | Buffer tersisa: 0 item
Producer menghasilkan: 86 | Buffer berisi: 1 item
Consumer memproses: 86 | Buffer tersisa: 0 item
Producer menghasilkan: 77 | Buffer berisi: 1 item
^C
nelsan@NELSAN:~$
```

Kesimpulan

Simulasi ini menunjukkan bahwa **mutex dan condition variable** sangat penting untuk menghindari **race condition** pada akses buffer bersama.

Dengan mekanisme ini, producer dan consumer dapat bekerja **sinkron dan aman**, tanpa saling menimpa data.

KESIMPULAN AKHIR

Dari ketiga tugas praktikum yang telah dilakukan, dapat diambil beberapa kesimpulan penting sebagai berikut:

1. Penerapan Thread dalam Kalkulator Paralel

Pada tugas pertama, penggunaan **multithreading** terbukti mampu menjalankan beberapa operasi aritmatika secara **paralel** (penjumlahan, pengurangan, perkalian, dan pembagian) dengan efisien. Setiap thread bekerja secara independen untuk satu jenis operasi, kemudian hasil dikumpulkan setelah semua thread selesai bekerja.

Hal ini menunjukkan bahwa dengan memanfaatkan konsep **paralelisme**, program dapat menyelesaikan beberapa pekerjaan dalam waktu yang relatif lebih singkat tanpa perlu menunggu proses lain selesai terlebih dahulu.

2. Efisiensi Pengolahan File dengan Multi-threading

Pada tugas kedua, percobaan menghitung jumlah kata dari sebuah file menunjukkan perbedaan waktu antara **single-thread** dan **multi-thread**.

- **Single-thread** memproses seluruh isi file secara linear dari awal sampai akhir.
- **Multi-thread** membagi file menjadi dua bagian yang dikerjakan secara bersamaan.

Dari hasil pengujian, waktu eksekusi metode **multi-thread** lebih cepat dibandingkan dengan **single-thread**, terutama pada file berukuran besar.

Hal ini menunjukkan bahwa **pembagian beban kerja antar-thread** dapat meningkatkan performa program dengan memanfaatkan kemampuan prosesor yang memiliki banyak inti (core). Namun demikian, pada file yang kecil, keuntungan tersebut tidak terlalu signifikan karena adanya overhead pembuatan dan sinkronisasi thread.

3. Sinkronisasi dan Komunikasi Antar-thread pada Producer–Consumer

Tugas ketiga memperlihatkan pentingnya **sinkronisasi** dalam komunikasi antar-thread yang bekerja pada data bersama.

Dalam simulasi **Producer–Consumer**, kedua thread mengakses buffer yang sama untuk menghasilkan dan mengonsumsi data.

Agar tidak terjadi konflik seperti **race condition**, digunakan mekanisme:

- **Mutex (Mutual Exclusion)** untuk mengamankan akses terhadap buffer.
- **Condition Variable** untuk mengatur alur kerja — producer menunggu jika buffer penuh, dan consumer menunggu jika buffer kosong.

Dengan demikian, kedua thread dapat bekerja secara **terkoordinasi dan aman**, memastikan integritas data tetap terjaga dan proses berjalan efisien tanpa deadlock atau data corruption.